

Önálló labor beszámoló

Játékfejlesztés Unity 3D-ben

Feladatomban az önálló labor alatt egy procedurálisan generált játékvilág előállításáról volt szó egy autós versenyjátékhoz. Az elképzeléseink viszonylag tiszták voltak már a fejlesztés elején is: az autók úton közlekednek, az utak mellett általában a természet honol, változatos élővilággal gazdagítva. Ezen alapvetések után egyszerű volt az elsőre komplex feladat felbontása:

- pályagenerálás
- domborzatgenerálás
- környezet benépesítése

A következőkben ezen lépések megvalósításának rögzítését mutatom be.

Pályagenerálás

Minden mérnöki feladatnál az első lépés a követelmények meghatározása, kialakítása, így én is ezzel kezdtem. Különböző utakat és versenypályákat vizsgálva egyszerűen rájöhettünk, hogy mindegyik alapja egy folytonos görbe, ami jó kiindulási pont a fejlesztéshez. Emellett fontos volt a szabályozhatóság, hogy a játékos mindig kedvének megfelelő pályán játszhasson. Célom volt a pályát 3 dimenzióssá tenni, így sokkal változatosabb és élvezetesebb vonalvezetések születhetnek.

Az alapgondolat pálya felépítésénél az volt, hogy generálok X darab egymáskövető pontot valamilyen algoritmussal, utána ezeket a pontokat kontrollpontként felhasználva készítek egy Catmull-Rom görbét a folytonosság miatt, majd ez alapján egy mesh hálót építek, ami pedig a végleges pályaként szolgál.

Ezek a lépések viszonylag jól is sikerültek, de utánuk több problémába is beleütköztem. Először teljesen randomizáltan generáltam a pályát, ami használhatatlan eredményt adott. A következő próbálkozásnál ezt kijavítva fixáltam a haladást az egyik tengelyen, ami viszont nem adott változatos eredményeket. A megoldás erre a problémára a determinisztikus iránygenerálás lett, ami a korábbi irányt elforgatva halad tovább. Ezzel nem csak a változatosságot sikerült megoldani, hanem a kanyargósság egyszerű állítására is lehetőség nyílik: csak felszorozom a forgatási szöveget az értékkel, és kész.

Éles kanyarnál előfordult, hogy a mesh nem tudta szépen lekövetni a belső ívet, és így láthatatlan akadályokat/glitch-et hozott létre a pályafalban, ami egyértelműen rontja a játékelményt. Ennek orvoslására két dolgot is sikerült találni: először is kanyarokban a középvonalat közelebb viszem a belső ívhez, ezáltal egy simább mesh hálót létrehozva. Viszont ez sem volt minden esetben megfelelő, emiatt a passzív helyett aktív módszerekkel eredtem a probléma után: a generált „falakon” végigmegyek, és ellenőrzöm, hogy meteszenek-e a

korábban generált részekkel. Amennyiben igen, akkor az újabb pontot áthelyezem a régi helyére [1.forrás, 73. oldal].

A harmadik, és talán legnehezebb problémám a pálya helyes generálásával volt. Ugyanis bizonyos esetekben (eltekintve az alapvetően helytelen első kontrollpont-generáló próbálkozásoktól) a pálya önmagába hurkolt. Az Y-tengely (magasság) használata miatt elsőre nem feltétlenül lenne ez probléma, el lehetne őket vinni egy alagúton vagy hídon keresztül, de ezek komplexitása és az ML ellenfelek által igényelt kontroll mesh (magas falak a pálya körül, amiket emelkedők és lejtők közben is tudnak észlelni) miatt nem foglalkoztam ezzel a lehetőséggel. Elsőként helyben próbáltam orvosolni a problémát, ami következőképp működött: egy új kontrollpont generálásánál leellenőriztem, nincs-e túl közel valamelyik régi ponthoz; amennyiben igen, úgy elforgattam az irányvektort ellentétes irányba, hogy eltávolítsam tőle. Viszont ezt a módszert is sikerült megfelelő paraméterekkel olyan helyzetbe hozni, hogy ne tudja előállítani a kívánt távolságot, ezáltal egy nem játszható pályát létrehozva.

Éreztem, hogy egy másik megközelítésre van szükségem, de nem sikerült utat találnom a bonyolultabbnál bonyolultabb matematikai bizonyítások között. A megoldást végül egy Ádámmal való beszélgetés hozta el: miután kifejtettem a problémám, rövid tanakodás után felvetette, hogy oldjam meg informatikusként a matematikai problémát. Ha egy hibás pályát generálok, akkor eldobom, és ugyanaz a seed alapján újragenerálok, és ezt folytatom ez első helyes pályáig.

Domborzatgenerálás

Alapos információgyűjtés és néhány stratégiai megbeszélés után tisztán kezdtek látszani a domborzattal kapcsolatos alapvetések: értelemszerűen a pálya körül kell lennie, ahhoz illeszkednie, optimalizálási célból cellákra javallott felosztani és valamilyen módon magasságokat kell generálnom a mesh megalkotásához.

Először kiszámolom a pályát befoglaló téglalap sarkait (plusz fix távolság, hogy az egész pálya körül legyen vége), majd előre meghatározott méretű (jelenleg 64x64, ez később fontos lesz) szektorokra bontom. Ezután a túl messze levő szektorokat, amik játék közben egyéb szektorok miatt nem tudnak vizuális élményt nyújtani, egyszerűen eldobom és nem generálok le őket.

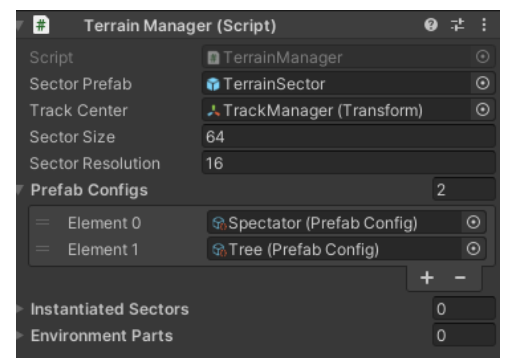
A heightmap létrehozására két módszert vizsgáltam meg: a Perlin zajt és a Diamond Square algoritmust. A Perlin zaj nevéből adódóan egyfajta zaj, viszont rendelkezik egy speciális tulajdonsággal: folytonos. Ez annyit tesz, hogy ha kiterítjük egy felületre és monokróm színskálájához [0:1] tartományt hozzárendeljük, egy folytonos domborzatot kapunk, nem pedig tüskéket. Ez alkalmassá teszi egyszerű, de mégis változatos domborzatok generálására. Itt érdemes megjegyezni, hogy a mi felhasználásunknál ez annyira nem mérvadó, mivel a stílusirányzat és optimalizációs okokból alacsony mintavételezéssel dolgozunk, így kapjuk meg a játékban található egyedi domborzatot.

A másik módszer, a Diamond Square egy kicsit kifinomultabb: egy $2n+1$ oldalméretű négyzetből indul, és 2 egyszerű lépést ismételve határozza meg a szektor magasságát. A két lépés az algoritmus nevéből adódóan a gyémánt és a négyzet lépés. A kiindulási állapotban a négyzet sarokpontjainak magassága van határozva. A gyémánt lépésben minden előállított négyzet középpontjának magassága a sarokpontok magasságának átlaga plusz egy random érték. Ehhez hasonlóan a négyzet lépésben minden gyémánt középpontjának magassága a sarokpontok átlaga plusz random érték. A véletlenszerű értéket többféleképpen is meghatározhatjuk, például Perlin zajjal is. Összeségében ez a módszer véleményem szerint egy szebb domborzatot hoz létre, így végül ezt is használjuk jelenleg.

Környezet benépesítése

Utolsó feladatomban a környezet megtöltése, életre keltése volt. Hogy ezt elérjem, szükséges volt megvizsgálnom, hogy az általunk elképzelt játékvilágban mi lenne helytálló. A játék műfaját és stílusát ismerve két egyszerűbb dizájnelem mellett tettem le a voksomat: a vaporwave esetében elengedhetetlen pálmafa és a versenyjátékokban megszokott nézőket.

A környezeti elemek egyszerűen bővíthetők az Unity Editor-ból, a megfelelő listában (Prefab Configs) egy új elemet létrehozva és néhány egyszerű paramétert megadva nem szükséges kódot módosítani.



Itt is két megközelítést vizsgáltam meg közelebbről: a „szimpla” véletlenszerű elhelyezést és a Voronoi-cellás megvalósítást. Az első nagyon egyszerűen működik: az adott elem generálódásának van egy fix esélye, és ha az adott pontra ez bekövetkezik, ott generálódik egy elem.

A Voronoi-cellák már egy kicsit bonyolultabban működnek: a szektoron belül generálok valahány seedpontot, amiket később „organikus” cellák meghatározására használok. Ennek lényege, hogy a cellán belül minden pontot a hozzá legközelebb található seedponthoz rendelem, az így létrejövő cellák pedig specifikálhatóak biomként.

Felhasznált hasznos források:

- [Jonas Freiknecht doktori disszertációja a procedurális generálásról](#)
- [Christopher McCooey diplomamunkája a procedurális generálásról](#)
- [Sebastian Lague videósorozata a procedurális környezetgenerálásról](#)
- [Sebastian Lague videósorozata az Unity-n belüli görbegerálásról](#)
- [Brackeys videója a procedurális terepgenerálásról](#)